Machine Learning — Lecture 1

Machine Learning Foundations and Generalization

FCIS — For Course Use Only

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 1 at a Glance

FCIS — For Private Use Only

Machine Learning Foundations and Generalization

Course Learning Outcome

CLO1: Explain the fundamental concepts of machine learning, including learning paradigms, data representation, model assumptions, and evaluation principles.

Lecture purpose

- Establish a precise vocabulary for **learning from data**.
- Distinguish the major learning paradigms and prediction task types.
- Represent datasets, models, parameters, and learning objectives mathematically.
- Explain why **generalization to unseen data** is the central goal of machine learning.

Conceptual structure

Learning phase	Core question
1. Machine-learning foundations	What does it mean to learn from experience, and what learning setting do we have?
2. Representation and learning	How do we express examples, features, targets, models, and learning objectives?
3. Generalization and complexity	Why can a model fit observed data yet fail on new data?
4. Reliable evaluation	How do we select and evaluate models without contaminating the evidence?

FCIS - For Course Use Only

💡 Main idea: machine learning is not the goal of fitting the training data perfectly; it is the goal of learning **useful patterns that generalize beyond the training examples**.

Learning Objectives

FCIS — Course Use Only

By the end of Lecture 1, you will be able to:

1. Explain machine learning using the relationship between **task, experience, and performance**.
2. Distinguish supervised, unsupervised, and reinforcement learning.
3. Distinguish regression, classification, clustering, and sequential decision problems.
4. Represent a dataset using observations, features, targets, a feature matrix X, and a target vector y.
5. Distinguish model parameters from hyperparameters and explain the role of a loss function.
6. Explain model assumptions and why representation affects what can be learned.
7. Explain training, validation, testing, generalization, underfitting, and overfitting.
8. Identify data leakage and describe sound evaluation principles for unseen data.

For Course Use Only

Lecture Roadmap

FCIS — Course Use Only

From learning from experience to reliable generalization

Learning phase	Part	Core question
1. Machine-learning foundations	**I**	What does learning from experience mean, and is the problem supervised, unsupervised, or reinforcement learning?
2. Representation and learning	**II**	How do we represent data, targets, models, parameters, hyperparameters, and learning objectives?
3. Generalization and complexity	**III**	Why can a model fit training data well yet perform poorly on unseen data?
4. Reliable evaluation	**IV**	How do we select, validate, and test models without leakage or test-set contamination?

FCIS — For Course Use Only

💡 **Practice & consolidation:** connect the full vocabulary, diagnose learning behavior, identify common mistakes, and justify evaluation decisions.

Part I

FCIS — For Students Use Only

Machine Learning Foundations and Learning Settings

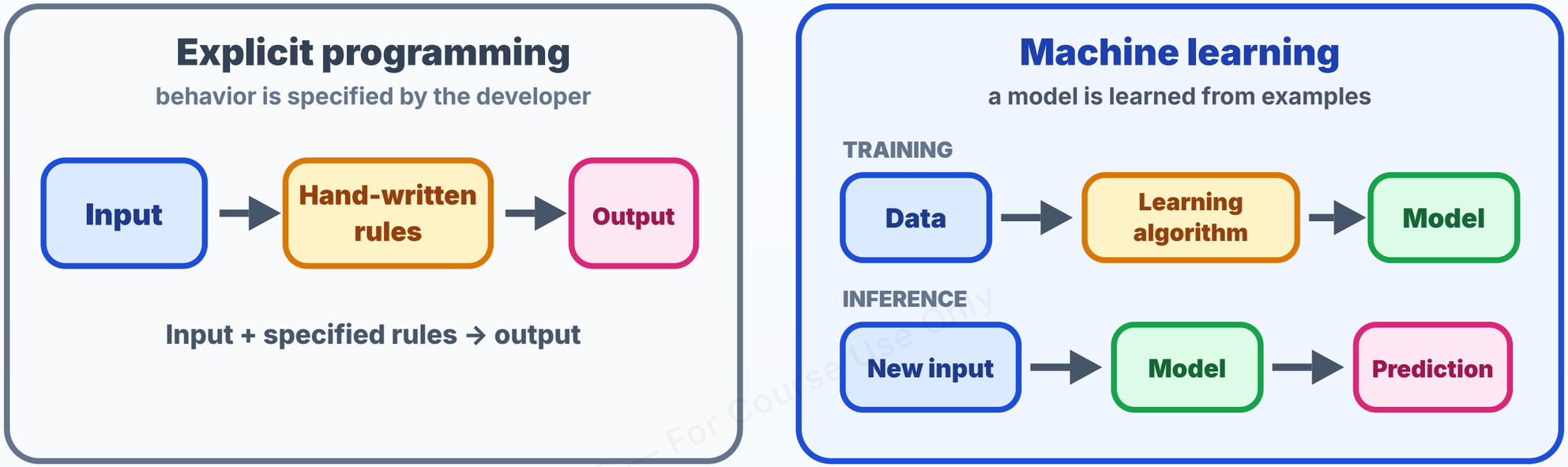
Machine learning studies methods that improve their performance by extracting **useful regularities from data or experience**.

FCIS — For Course Use Only

From Explicit Programming to Learning

For Course Only

Two ways to build a system



FCIS – For Course Use Only

💡 **Key idea.** Explicit programming specifies the behavior directly. Machine learning uses examples to **learn a model**, which is then applied to **new inputs**.

A Practical Definition of Learning

FCIS — For Educational Use Only

Task, experience, and performance

A useful operational description of a learning system involves three elements:

$$\text{Task } T, \qquad \text{Experience } E, \qquad \text{Performance } P$$

A system **learns** when experience E leads to improved performance P on task T.

Example — Email spam detection

Element	Example
Task T	Classify an email as spam or not spam
Experience E	Previously labeled email messages
Performance P	Quality of predictions on unseen email

The exact performance measure depends on the application. “Good performance” is not always the same as “high accuracy.”

FCIS — Course Use Only

When Is Machine Learning Useful?

FCIS — Classroom Use Only

ML is valuable when useful behavior can be learned from evidence

Situation	Why learning helps	Example
Rules are hard to specify	Patterns are easier to observe than to encode manually	Fraud detection
The environment changes	The model can be updated using newer observations	Demand forecasting
Many interacting variables matter	A model can estimate relationships from examples	House-price prediction
Personalization matters	Behavior can adapt to users	Recommendation systems
Structure is unknown	Unsupervised methods can reveal patterns	Customer segmentation

FCIS — For Course Use Only

Machine learning is not automatically appropriate. If a task has exact, stable, and easily specified rules, explicit programming may be simpler, safer, and easier to verify.

Machine Learning Is More Than Prediction

For Course Use Only

Learning problems can have different objectives

Objective	Typical question	Example output
Predict a number	What value should we expect?	House price: 850,000 SAR
Predict a category	Which class does this example belong to?	Fraud / legitimate
Discover groups	Which observations behave similarly?	Customer segments
Learn a representation	Can data be expressed more compactly?	Principal components
Choose actions over time	Which action maximizes long-term reward?	Robot navigation policy

DIS — For Course Use Only

> The type of output and the available feedback are two of the first clues for identifying the appropriate **learning paradigm**.

Learning Paradigms

FCIS — Course Use Only

The central distinction is the kind of feedback available

Paradigm	Training information	Main objective	Typical example
Supervised learning	Inputs with desired outputs	Learn a mapping from inputs to targets	Predict loan default
Unsupervised learning	Inputs without target labels	Discover structure in the data	Segment customers
Reinforcement learning	Actions followed by rewards or penalties	Learn behavior that maximizes long-term reward	Learn robot navigation

FCIS — For Course Use Only

💡 The algorithms differ, but all three paradigms use experience to improve future behavior or representation.

Supervised Learning

FCIS — Private Use Only

Learning from input–output examples

A supervised dataset contains examples of the form

$$(x_i, y_i),$$

where:

- x_i describes the input features for example i,
- y_i is the desired output or **target**.

The objective is to learn a function

$$h_\theta(x) \approx y$$

that also performs well for **new examples** not used during training.

> Given historical information about customers and whether they defaulted, learn to predict the default risk of a new customer.

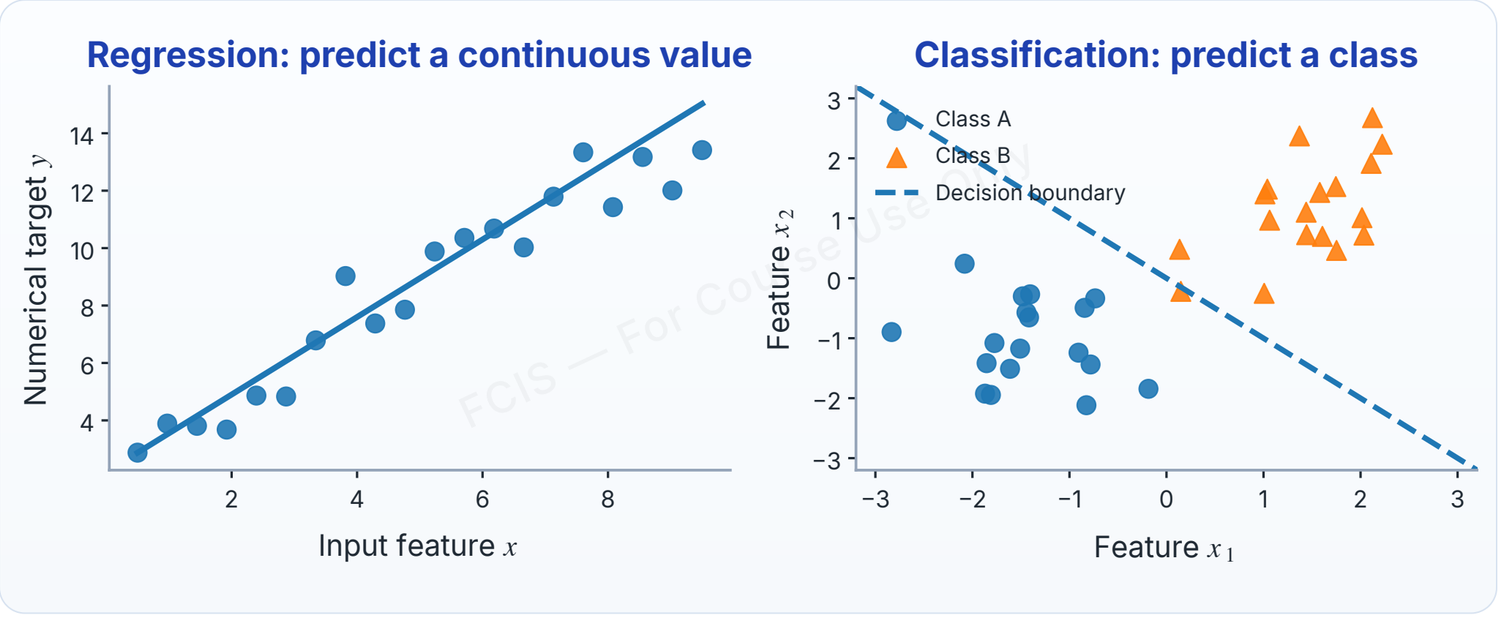
FCIS — For Course Use Only

Regression vs. Classification

FCIS — For Course Use Only

Both are supervised, but the output type differs

Task	Target	Typical question
Regression	Numerical / continuous	What value should be predicted?
Classification	Category / class	Which class is most appropriate?



The distinction depends on the **target variable**: regression predicts a numerical value, while classification predicts a category. The input features themselves may be numerical or categorical in either task.

Unsupervised Learning

FCIS — For Course Use Only

Learning when no target y is provided

In unsupervised learning, the dataset is typically represented only by inputs:

$$D = \{x_i\}_{i=1}^{n}.$$

The goal is not to match known labels. Instead, we seek useful **structure** in the data.

Common objectives

- **Clustering:** group similar observations.
- **Dimensionality reduction:** represent high-dimensional data using fewer variables.
- **Structure discovery:** reveal patterns or relationships that were not explicitly labeled.

FCIS - For Course Use Only

> A retailer may cluster customers according to spending and ordering behavior without defining customer types in advance.

Reinforcement Learning

FCIS — For Reference Use Only

Learning through interaction and consequences

A reinforcement-learning agent repeatedly interacts with its environment:

State
Current situation

→

Action
Choice made by the agent

→

Reward
Numerical feedback

→

New State
Updated situation after acting

The objective is to learn a strategy that maximizes **cumulative reward**, not only the reward of the next action.

Element	Meaning
State	Current situation available to the agent
Action	Choice made by the agent
Reward	Numerical feedback after an action
Policy	Rule for selecting actions

FCIS – For Course Use Only

💡 This course uses **reinforcement learning** mainly to complete the conceptual map of learning paradigms. The core algorithms studied later focus on **supervised** and **unsupervised learning**.

Learning-Setting Activity

FCIS — For Educational Use Only

Identify the paradigm and task type

Scenario	Paradigm?	Task type?
Predict monthly electricity consumption from historical records	?	?
Discover groups of patients with similar measurements	?	?
Predict whether a transaction is fraudulent	?	?
A robot learns actions using rewards for reaching a destination	?	?
Reduce 100 correlated measurements to a small set of components	?	?

FCIS — For Course Use Only

Identify the paradigm and task type

Scenario	Paradigm?	Task type?
Predict monthly electricity consumption from historical records	?	?
Discover groups of patients with similar measurements	?	?
Predict whether a transaction is fraudulent	?	?
A robot learns actions using rewards for reaching a destination	?	?
Reduce 100 correlated measurements to a small set of components	?	?

FCIS — For Course Use Only

Suggested classification: supervised–regression; unsupervised–clustering; supervised–classification; reinforcement learning; unsupervised–dimensionality reduction.

Part II

FCIS — For Private Use Only

Data Representation and the Learning Problem

Before an algorithm can learn, the real-world problem must be converted into **examples, features, targets, and a model representation**.

FCIS — For Course Use Only

From Real-World Objects to Data

FCIS — For Educational Use Only

A dataset is an abstraction of the problem

Suppose we want to predict apartment prices.

Apartment	Area (m^2)	Rooms	Age	Neighborhood	Price (SAR)
A	120	3	5	Central	720,000
B	90	2	12	North	510,000
C	160	4	2	Central	980,000
D	105	3	8	East	640,000

FCIS — For Course Use Only

Here:

- each **row** is an observation or example,
- input columns are **features**,
- `Price` is the **target**.

> A representation determines what information the model can see. Important real-world information that is omitted from the representation cannot be used by the model.

Observations, Features, and Targets

FCIS - For Course Use Only

Core vocabulary

Term	Meaning	Apartment example
Observation / sample	One example in the dataset	One apartment
Feature / predictor	Input variable used to make a prediction	Area, rooms, age
Target / response	Quantity or class we want to predict	Price
Feature vector x_i	Feature values for one observation	$[120, 3, 5, \ldots]$
Dataset size n	Number of observations	Number of apartments
Number of features p	Number of input variables	Number of predictors

For Course Use Only

The target is part of the training information in supervised learning, but it is not normally available when predicting a genuinely new case.

Matrix Representation

FCIS — Private Use Only

Machine-learning notation

For n observations and p features, the feature matrix is

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p}$$

For supervised learning, targets are often collected as

$$y = [y_1, y_2, \ldots, y_n]^\top$$

FCIS — For Course Use Only

💡 Convention used in this course: **rows are observations; columns are features**.

Numerical and Categorical Features

For Course Use Only

Real datasets contain different data types

Feature type	Example	Typical representation issue
Continuous numerical	Area, temperature	Scale and distribution
Discrete numerical	Number of orders	Range and interpretation
Nominal categorical	Neighborhood	Must not invent an artificial order
Ordinal categorical	Low / medium / high	Ordered categories may carry rank information
Binary	Has loan? yes/no	Can often be represented by 0/1

FCIS — For Course Use Only

If `Neighborhood = {Central, North, East}`, coding them as 1, 2, 3 may incorrectly imply that East is “three times” Central or that distances between categories are meaningful.

💡 Lecture 2 studies encoding, scaling, missing data, transformations, and feature engineering in detail.

Feature Space

FCIS — Lecturer Use Only

Each observation occupies a point in a representation space

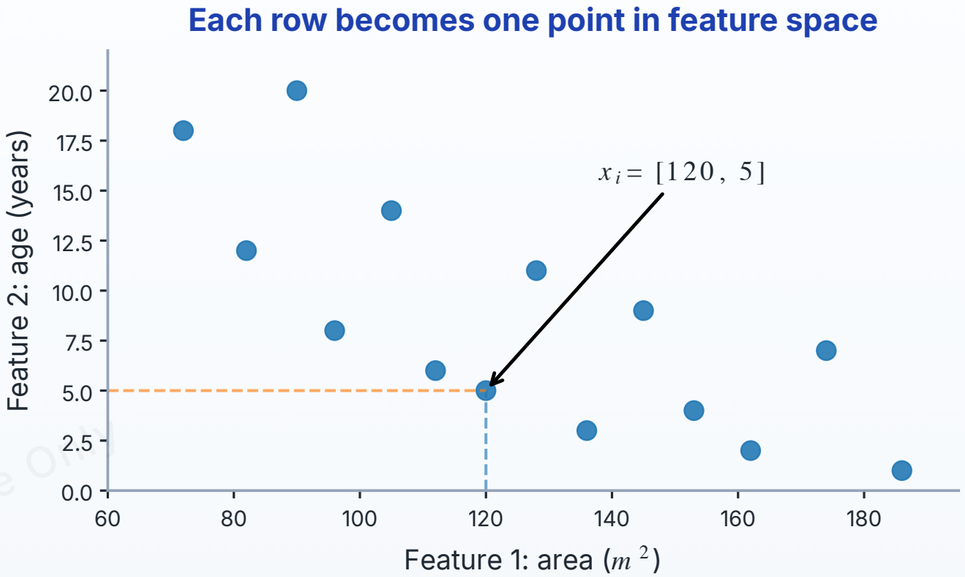
If two numerical features are used,

$$x_i = [x_{i1}, x_{i2}]$$

then each observation can be viewed as a point in a two-dimensional **feature space**.

With (p) features, the same idea extends to

$$x_i \in \mathbb{R}^p.$$



Why this matters

- **KNN** uses distance between points.
- Linear models learn boundaries or surfaces in this space.
- **K-Means** groups points around centroids.
- **PCA** changes the coordinate system to capture major directions of variation.

> How we represent a problem directly affects the **geometry** and information available to the learning algorithm.

FCIS — For Course Use Only

Representation Matters

FCIS — Private Use Only

Same real-world case, different machine-learning problem

Consider customer purchase history.

Representation	Possible learning question
Total spending, orders, days since last purchase	Predict future spending
Same features + target `Churn`	Predict churn class
Same features without target	Discover customer segments
Sequence of daily actions and rewards	Learn a retention policy

For Course Use Only

💡 The algorithm does not receive the full real-world customer. It receives the **representation we choose**.

Consequence

A poor representation can hide important information, introduce misleading structure, or make an otherwise easy learning problem unnecessarily difficult.

The Model or Hypothesis

FCIS — Faculty Use Only

A model maps a representation to an output

A supervised model can be written as

$$h_\theta : \mathcal{X} \to \mathcal{Y}$$

where θ denotes the model's adjustable **parameters**.

For example, a linear regression model may be

$$h_\theta(x) = w^\top x + b$$

FCIS — For Course Use Only

Here,

$$\theta = (w, b)$$

> The **model family** specifies what kinds of functions can be represented; learning selects a particular model from that family using the training data.

Parameters vs. Hyperparameters

FCIS — For Private Use Only

Both affect the model, but they are chosen differently

Concept	How it is obtained	Examples
Parameter	Learned from training data	Regression coefficients, tree split values, neural-network weights
Hyperparameter	Chosen outside the normal parameter-fitting step	Tree depth, number of neighbors k, regularization strength

For KNN, the training examples are stored, but the value of k is a **hyperparameter** that must be selected using a principled validation procedure.

Choosing a hyperparameter after repeatedly inspecting the final test results makes the test set part of the design process and invalidates its role as an unbiased final evaluation.

FCIS — For Course Use Only

What Does the Learning Algorithm Do?

FCIS — ML Course

It uses training data to select a model

For supervised learning, consider

$$D = \{(x_i, y_i)\}_{i=1}^{n}.$$

A loss function L measures the disagreement between prediction and target:

$$L(h_\theta(x_i), y_i).$$

The average training loss is

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} L(h_\theta(x_i), y_i).$$

FCIS — For Course Use Only

A common learning objective is

$$\theta^* = \arg\min_{\theta} \widehat{R}(\theta).$$

💡 Minimizing training loss is useful only if it leads to low error on **future observations**. This is the bridge from optimization to generalization.

Loss Functions Depend on the Task

FCIS — Personal Use Only

“Error” must be defined mathematically

Problem	Prediction	Possible loss
House-price regression	Numerical value	Squared error $(y - \hat{y})^2$
Binary classification	Probability / class	Log loss
Multiclass classification	Class probabilities	Cross-entropy
Clustering	Cluster assignment	Within-cluster distance objective

For Course Use Only

A **loss function** evaluates one prediction or model decision mathematically. An **evaluation metric** summarizes performance in a way suitable for judging a trained model.

💡 Loss and evaluation metric can be related without being identical. Later lectures distinguish the quantity optimized during training from the quantity used to judge deployment performance.

Training vs. Inference

FCIS — For Course Use Only

Two different phases of a learned system

Training phase

$$D_{train} \rightarrow \text{Learning Algorithm} \rightarrow \hat{\theta}$$

The algorithm uses examples to estimate parameters or structure.

Questions:

- Which model family?
- Which loss?
- Which hyperparameters?
- How should data be prepared?

Inference phase

$$x_{new} \rightarrow h_{\hat{\theta}} \rightarrow \hat{y}$$

The fitted model is applied to a new input.

> Training a fraud classifier may take minutes or hours; classifying one new transaction should often take milliseconds.

FCIS — For Course Use Only

Model Assumptions

FCIS — Private Use Only

Every model simplifies reality

A model is useful because it imposes **structure** on the learning problem.

Examples of assumptions include:

- a relationship can be approximated by a linear function;
- nearby observations tend to have similar outputs;
- a sequence of decision-tree splits can separate important cases;
- training examples are sufficiently representative of future cases;
- features contain enough information to predict the target.

FCIS — For Course Use Only

A **model assumption** is a restriction or belief about the relationship between data and outcomes that makes learning possible or manageable.

An assumption does not need to be perfectly true. It must be useful enough for the model to perform reliably for the intended task.

Inductive Bias

FCIS — Academic Use Only

Learning requires preferences among possible explanations

A finite training dataset is usually compatible with many possible functions.

The learning algorithm therefore needs an **inductive bias**: a preference that helps it choose among models that fit the observed data.

Examples

- Linear regression prefers linear relationships.
- KNN assumes nearby points should often behave similarly.
- A shallow decision tree prefers a simpler sequence of rules.
- Regularization prefers parameter values with controlled complexity.

For Course Use Only

💡 Without assumptions or preferences, observations from the past give no principled basis for predicting unseen cases.

Model Flexibility

FCIS — For Classroom Use Only

More flexible is not automatically better

Model behavior	Advantage	Risk
Low flexibility	Simple, stable, easier to interpret	May miss important structure
Moderate flexibility	Can capture useful patterns	Requires validation and tuning
Very high flexibility	Can fit complex relationships	May fit noise and accidental details

IUM — For Course Use Only

A very deep decision tree can often classify the training examples almost perfectly, yet perform worse on new observations than a smaller tree.

The central question is not “How well can the model fit the observed data?” but “How well does the learned pattern transfer to unseen data?”

Part III

FCIS — For Private Use Only

Generalization and Model Complexity

A model is useful when it performs well on **new observations drawn from the problem we actually care about**.

FCIS — For Course Use Only

Generalization

FCIS — For Private Use Only

Training performance is evidence, not the final objective

Let

$$R_{train}$$

represent error measured on training examples, and let

$$R_{new}$$

represent expected error on new observations.

We can directly observe R_{train}, but the real objective is to obtain low R_{new}.

Generalization is the ability of a learned model to perform well on data that were not used to fit the model.

Excellent training performance is not evidence of successful learning if the model fails on unseen examples.

ECIS Course Use Only

Why Training Error Is Optimistic

FCIS_ Private Use Only

The model has already seen the training set

During training, the model is selected specifically to reduce error on the observed examples.

Therefore:

$$R_{train}$$

usually gives an **optimistic estimate** of future performance.

Analogy

If students receive the exact exam questions in advance, high scores on those questions do not prove that they understand the subject well enough to solve new problems.

> A 1-nearest-neighbor classifier can achieve perfect training accuracy in many datasets by simply matching each observation to itself, yet its accuracy on new data may be substantially lower.

FCIS — For Course Use Only

Underfitting

FCIS — For Personal Use Only

The model is too limited to capture important structure

Typical signs:

- training error is high;
- validation error is also high;
- adding more flexibility may improve both;
- the model systematically misses important patterns.

FCIS For Course Use Only

Fitting a straight line to a strongly curved relationship may produce large errors on both training and validation data.

Underfitting often reflects **excessive simplification**, poor features, or insufficient learning capacity.

Overfitting

FCIS — Personal Use Only

The model learns accidental details of the training data

Typical signs:

- training error is very low;
- validation/test error is noticeably higher;
- the model is sensitive to small changes in training data;
- complexity captures noise rather than stable signal.

For Course Use Only

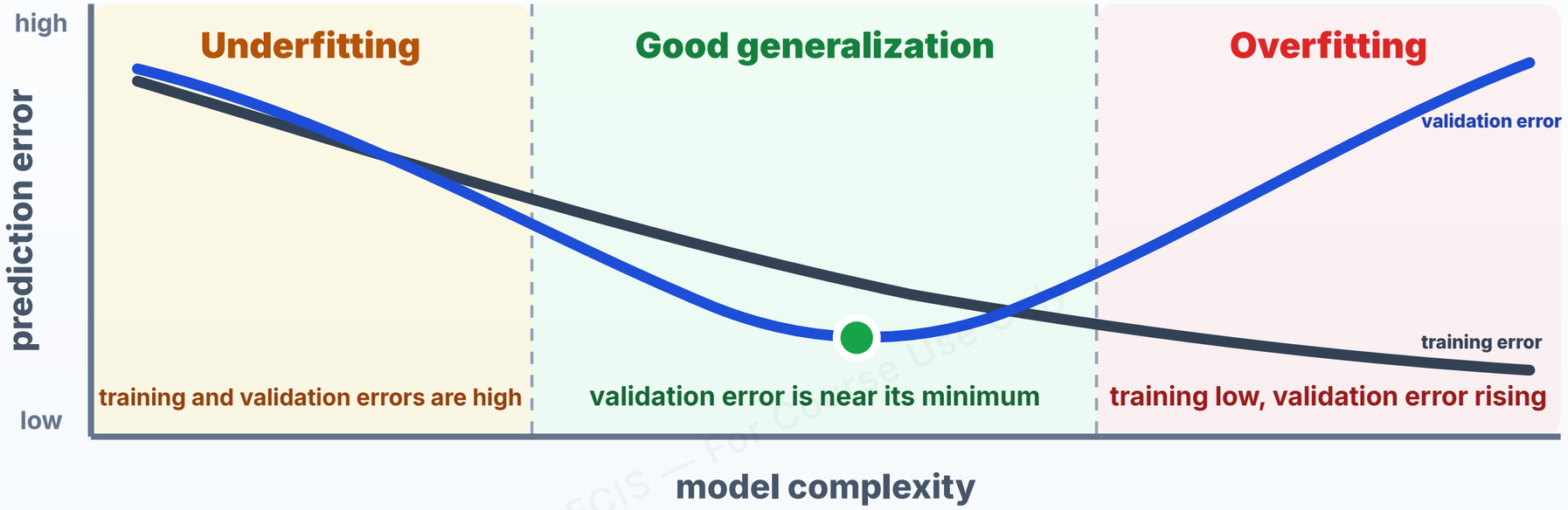
A deep decision tree may create many highly specific branches that classify all training observations correctly but generalize poorly.

Overfitting is not “too much training accuracy.” It is the **gap between fitting observed examples and performing on unseen examples**.

Underfitting vs. Good Fit vs. Overfitting

For Course Use Only

Training error alone is not enough



FCIS — For Course Use Only

💡 **Key idea.** As model complexity increases, **training error** usually decreases, but **validation error** may eventually rise if the model starts fitting noise. Model selection should therefore rely on **validation performance**, not training error alone.

Bias–Variance Intuition

FCIS — Course Use Only

Two reasons predictions can fail

High bias

- Model assumptions are too restrictive.
- Important structure is missed consistently.
- Often associated with **underfitting**.

Using a simple straight line for a strongly nonlinear relationship.

High variance

- Model changes strongly with the training sample.
- It captures unstable details and noise.
- Often associated with **overfitting**.

A highly complex tree built from a small dataset.

FCIS — For Course Use Only

💡 Model selection seeks useful predictive structure while controlling unnecessary sensitivity to the particular training sample.

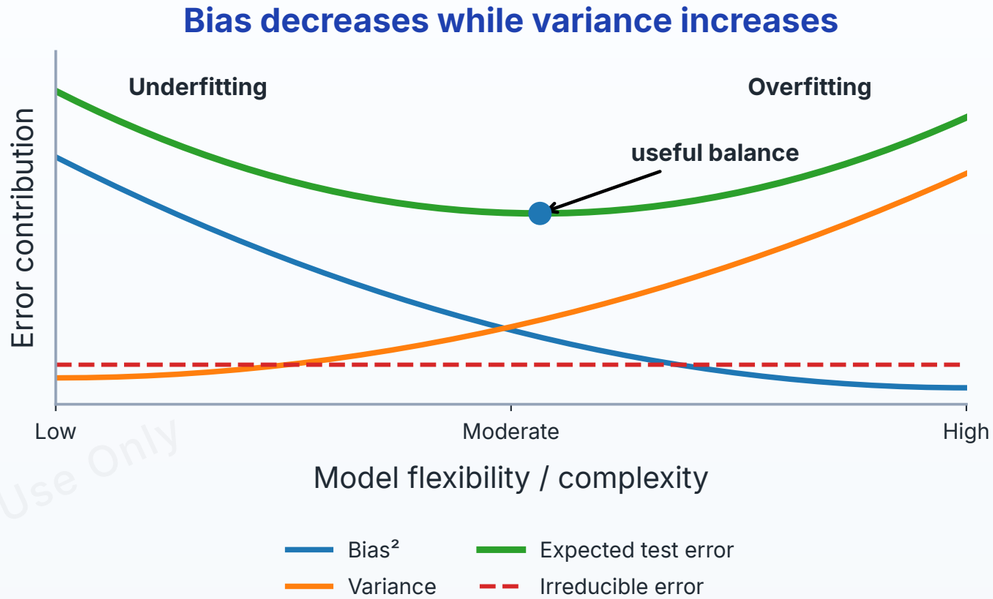
The Bias–Variance Trade-off

FCIS — For Reference Use Only

Complexity changes the kind of error we make

As model flexibility increases, a common qualitative pattern is:

- **Bias decreases** because the model can represent richer relationships.
- **Variance increases** because the fitted model can become more sensitive to the training sample.
- **Prediction error on unseen data** is often lowest at an intermediate level of flexibility.



FCIS — For Course Use Only

The goal is not to eliminate bias or variance completely. The goal is to choose a model whose **overall prediction error on unseen data is appropriately low**.

Important: this is a reasoning framework, not a rule for selecting complexity without validation data.

Noise and Irreducible Error

FCIS — For Course Use Only

Some uncertainty cannot be removed by a better model

Observed outcomes may depend on:

- unmeasured variables;
- measurement error;
- random events;
- changes in the environment;
- inherently unpredictable behavior.

A useful conceptual decomposition is

$$\text{Prediction Error} = \text{Model-related Error} + \text{Irreducible Error}$$

A more complex model cannot recover information that is absent from the data. Trying to fit irreducible noise can actually worsen generalization.

FCIS — For Course Use Only

Part IV

FCIS — Personal Use Only

Reliable Model Selection and Evaluation

Reliable evaluation requires separating **model fitting**, **model selection**, and **final testing**, while preventing information from leaking across those roles.

FCIS — For Course Use Only

Train, Validation, and Test Sets

FCIS — Internal Use Only

Different data partitions serve different purposes

Partition	Main role	Used to choose the model?
Training set	Fit parameters or model structure	Yes — through learning
Validation set	Compare models and select hyperparameters	**Yes**
Test set	Estimate final performance after model selection	**No**

CIS — For Course Use Only

A common conceptual split is

$$D = D_{train} \cup D_{val} \cup D_{test}$$

with disjoint subsets.

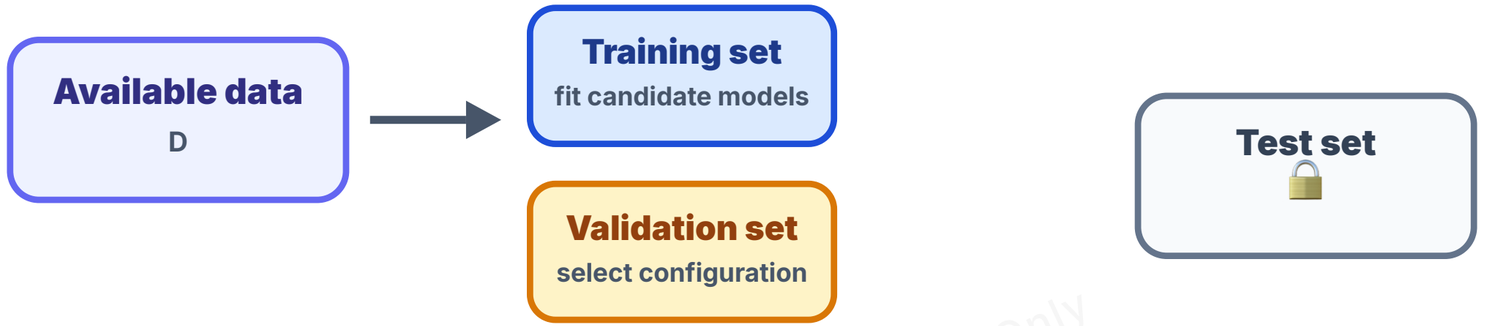
> The exact percentages are problem-dependent. The important principle is the **separation of roles**, not a universal 60/20/20 rule.

Correct Model-Selection Protocol

FCIS — For Class Use Only

Keep the final test set outside the design loop

Step 1 — split the data before model selection



Training and validation may influence design decisions; the test set must not.

FCIS — For Course Use Only

The test set approximates future unseen data only while it remains **independent of model-selection decisions**.

💡 Later, cross-validation will provide a more data-efficient way to perform model selection when a single validation split is not sufficient.

Keep the final test set outside the design loop

Step 2 — fit candidate models using training data



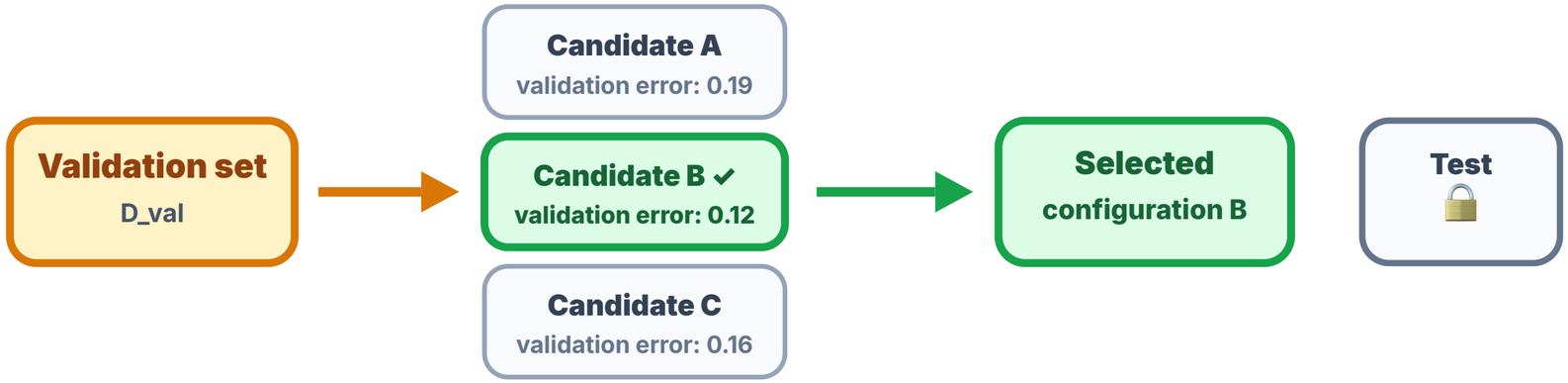
Parameters are fitted using training data only.

The test set approximates future unseen data only while it remains **independent of model-selection decisions**.

💡 Later, cross-validation will provide a more data-efficient way to perform model selection when a single validation split is not sufficient.

Keep the final test set outside the design loop

Step 3 — use validation performance to select the configuration



Validation data may guide model and hyperparameter selection.

The test set approximates future unseen data only while it remains **independent of model-selection decisions**.

💡 Later, cross-validation will provide a more data-efficient way to perform model selection when a single validation split is not sufficient.

Keep the final test set outside the design loop

Step 4 — evaluate the selected model once on the test set



Do not return to model tuning after inspecting the test result.

Optional: refit the chosen configuration on the available non-test training data before this final evaluation.

The test set approximates future unseen data only while it remains **independent of model-selection decisions**.

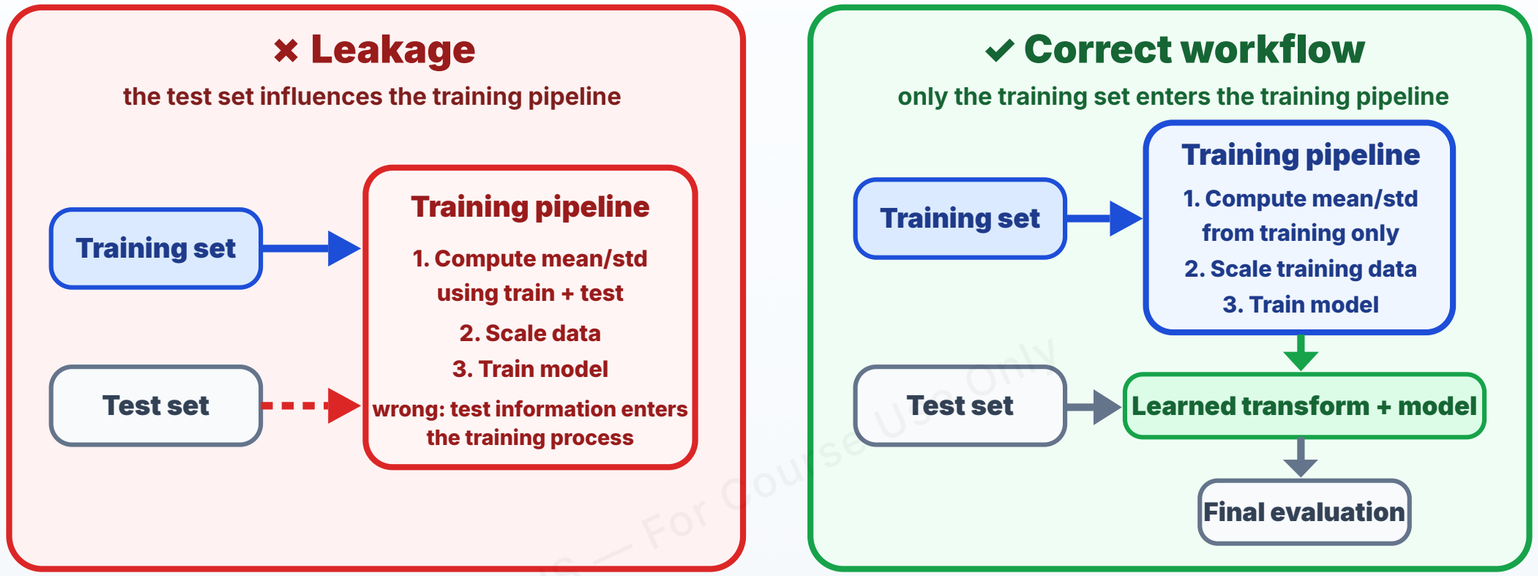
💡 Later, cross-validation will provide a more data-efficient way to perform model selection when a single validation split is not sufficient.

Data Leakage

FCIS — For Course Use Only

Keep the test set out of the training pipeline

Data leakage occurs when information from outside the legitimate training process influences preprocessing, model fitting, or model selection.



Test information influences how training is performed. The test set uses the learned transform; it does not help learn it.

Concrete example. If annual income is standardized, compute μ_{train} and σ_{train} from the **training set only**, then apply the same transformation to validation and test data: $x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$

Leakage also occurs when we select features using the full dataset, tune hyperparameters after inspecting the test result, or let duplicate/future information cross the train–test boundary.

Evaluation Principles

FCIS — Educational Use Only

A valid score requires a valid evaluation design

Principle	Why it matters
Evaluate on unseen data	Measures generalization rather than memorization
Match the metric to the task	Different errors have different meanings
Use validation for model selection	Prevents test-set contamination
Compare against a baseline	Shows whether the model adds useful predictive value
Preserve realistic data conditions	Evaluation should resemble deployment
Interpret uncertainty and trade-offs	A single score rarely tells the whole story

FCIS — For Course Use Only

💡 Lecture 5 develops these ideas quantitatively using regression metrics, confusion matrices, precision, recall, F1, validation, and model-selection procedures.

Baselines

FCIS — Use Only

A model should beat a meaningful simple reference

A **baseline** is a simple reference strategy used to judge whether a learned model is genuinely useful.

Examples

- Regression: always predict the training-set mean or median.
- Classification: always predict the majority class.
- Time series: use the most recent observed value as a naive forecast.

If 98% of transactions are legitimate, a classifier that always predicts “legitimate” obtains 98% accuracy while detecting **zero fraud cases**.

A high score has little meaning without context. Baselines reveal whether the model improves over a simple alternative.

FCIS — For Course Use Only

Evaluation Must Match the Real Decision

Please For Course Only

Different mistakes can have different consequences

Consider two errors in fraud detection:

- **False positive:** legitimate transaction flagged as fraud.
- **False negative:** fraudulent transaction accepted as legitimate.

These errors usually have different operational costs.

Similarly:

- missing a serious medical condition can be more costly than ordering an extra test;
- falsely rejecting a loan applicant has different consequences from approving a high-risk loan;
- a 10,000 SAR house-price error may be small for one property and large for another.

ECIS - For Course Use Only

💡 Model evaluation is ultimately a **decision problem**: we need metrics and thresholds that reflect the consequences of errors in the real context.

Worked Scenario: House-Price Prediction

For Course Usage Only

Connect the vocabulary

A real-estate company has historical records containing `Area`, `Rooms`, `Age`, `Neighborhood`, and `SalePrice`.

Question	Interpretation
What is the paradigm?	**Supervised learning**
What is the task type?	**Regression**
What is one observation?	One sold property
What are the features?	Area, rooms, age, neighborhood
What is the target?	Sale price
What must happen to neighborhood?	It requires an appropriate numerical representation
What does training do?	Estimates a model from historical examples
What does validation do?	Helps choose model/hyperparameters
What does testing do?	Estimates final generalization performance

FCIS — For Course Use Only

> A model with slightly higher training error can still be preferable if its validation error is lower and more stable.

Worked Scenario: Fraud Detection

FCIS — For Course Use Only

Why evaluation design matters

Suppose a dataset contains transaction features and a target:

$$y \in \{\text{fraud}, \text{legitimate}\}.$$

A model achieves:

- 99% training accuracy;
- 98.5% validation accuracy;
- but 98.7% of all transactions are legitimate.

Questions

1. Is the model necessarily useful?
2. What baseline should we check?
3. Are all classification errors equally costly?
4. Why should the test set remain untouched until the end?

FCIS — For Course Use Only

Why evaluation design matters

Suppose a dataset contains transaction features and a target:

$$y \in \{\text{fraud}, \text{legitimate}\}.$$

A model achieves:

- 99% training accuracy;
- 98.5% validation accuracy;
- but 98.7% of all transactions are legitimate.

Questions

1. Is the model necessarily useful?
2. What baseline should we check?
3. Are all classification errors equally costly?
4. Why should the test set remain untouched until the end?

FCIS — For Course Use Only

Key insight: accuracy alone may hide failure on the rare fraud class. A majority-class baseline is already very strong, so later we will need metrics that examine different error types explicitly.

Practice & Consolidation

FCIS — For Course Use Only

Reasoning About Complete Machine-Learning Problems

The goal now is to connect the lecture concepts and justify decisions about the **learning setting, representation, generalization, and evaluation**.

FCIS — For Course Use Only

Reasoning Activity

FCIS — Internal Use Only

Diagnose the learning problem

A student builds three models for the same classification task:

Model	Training Accuracy	Validation Accuracy
A	72%	70%
B	91%	88%
C	100%	76%

Answer before revealing the interpretation:

1. Which model is most likely underfitting?
2. Which model shows the strongest sign of overfitting?
3. Which model would you tentatively select?
4. Is validation accuracy alone enough for every real application?

For Course Use Only

Diagnose the learning problem

A student builds three models for the same classification task:

Model	Training Accuracy	Validation Accuracy
A	72%	70%
B	91%	88%
C	100%	76%

Answer before revealing the interpretation:

1. Which model is most likely underfitting?
2. Which model shows the strongest sign of overfitting?
3. Which model would you tentatively select?
4. Is validation accuracy alone enough for every real application?

FCIS - For Course Use Only

Suggested reasoning: A is the strongest underfitting candidate; C shows a large generalization gap; B is the most promising based on these values. The final decision may still depend on class balance, error costs, and the selected evaluation metric.

Common Conceptual Mistakes

FCIS — Course Use Only

Avoid these misunderstandings early

Mistake	Better interpretation
“ML means neural networks.”	Neural networks are one family among many ML methods.
“Unsupervised learning predicts labels.”	It usually discovers structure without predefined target labels.
“More features are always better.”	Irrelevant, redundant, or leaked features can harm reliability.
“100% training accuracy means the model is excellent.”	It may indicate severe overfitting.
“The test set is for tuning.”	Validation is for tuning; test data are for final evaluation.
“The most complex model is the best.”	Complexity must be justified by improved generalization.
“Accuracy is always the correct metric.”	The correct metric depends on the task and consequences of errors.

FCIS — For Course Use Only

Mini-Quiz

FCIS — For Course Use Only

Check your understanding

1. A system discovers groups of shoppers without predefined labels. Which learning paradigm is this?
2. In a house-price dataset, what is the difference between a feature and the target?
3. What is the difference between a parameter and a hyperparameter?
4. Why is training error usually an optimistic estimate of future error?
5. What role does the validation set play?
6. Give one example of data leakage.
7. Which pattern is more consistent with overfitting: high train/high validation error, or very low train/high validation error?

FCIS — For Course Use Only

Check your understanding

1. A system discovers groups of shoppers without predefined labels. Which learning paradigm is this?
2. In a house-price dataset, what is the difference between a feature and the target?
3. What is the difference between a parameter and a hyperparameter?
4. Why is training error usually an optimistic estimate of future error?
5. What role does the validation set play?
6. Give one example of data leakage.
7. Which pattern is more consistent with overfitting: high train/high validation error, or very low train/high validation error?

FCIS — For Course Use Only

Suggested answers: unsupervised learning; features are inputs while price is the target; parameters are learned while hyperparameters configure the learning/model; the model was optimized using the training examples; validation supports model selection; fitting preprocessing using test data is leakage; very low training error with substantially higher validation error suggests overfitting.

Lecture 1 Summary

FCIS — For Course Use Only

Key ideas to retain

Learning and representation

- Machine learning improves performance using data or experience.
- Supervised learning uses targets; unsupervised learning discovers structure; reinforcement learning uses reward feedback.
- Regression predicts numerical values; classification predicts categories.
- Data are represented using observations, features, targets, and matrices.
- Models encode assumptions and contain learned parameters.

Generalization and evaluation

- Training loss is not the final objective; unseen-data performance is.
- Validation supports model and hyperparameter selection.
- Test data should remain outside the model-selection loop.
- Underfitting and overfitting are failures of generalization.
- Good evaluation requires suitable metrics, realistic data, and meaningful baselines.

IUS — For Course Use Only

End-of-Lecture Check

FCIS — Private Use Only

Five questions to test your understanding

1. How do task T, experience E, and performance P describe a learning system?
2. Why can the same dataset support a regression, classification, or unsupervised problem depending on the target and objective?
3. What is the conceptual difference between a model parameter and a hyperparameter?
4. Why should preprocessing and model selection be performed without using information from the test set?
5. How can training and validation errors help distinguish underfitting from overfitting?

FCIS — For Course Use Only

💡 **Preparation for next lecture:** Review numerical vs. categorical features, summary statistics, quartiles, and the purpose of scaling. Lecture 2 studies data understanding, preprocessing, transformation, and feature engineering in depth.

Suggested Reading

FCIS — Private Use Only

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Chapter 1: Introduction
- Chapter 2: Statistical Learning
- Focus on the statistical learning framework, prediction vs. inference, model flexibility, training error, test error, and the bias–variance trade-off.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review the machine-learning landscape and the end-to-end concepts of data, training, validation, and testing.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected introductory sections for a deeper view of models, uncertainty, generalization, and probabilistic reasoning.

FCIS — For Course Use Only

💡 When reading, focus on the distinction between **fitting observed data** and **learning a pattern that performs reliably on unseen data**.